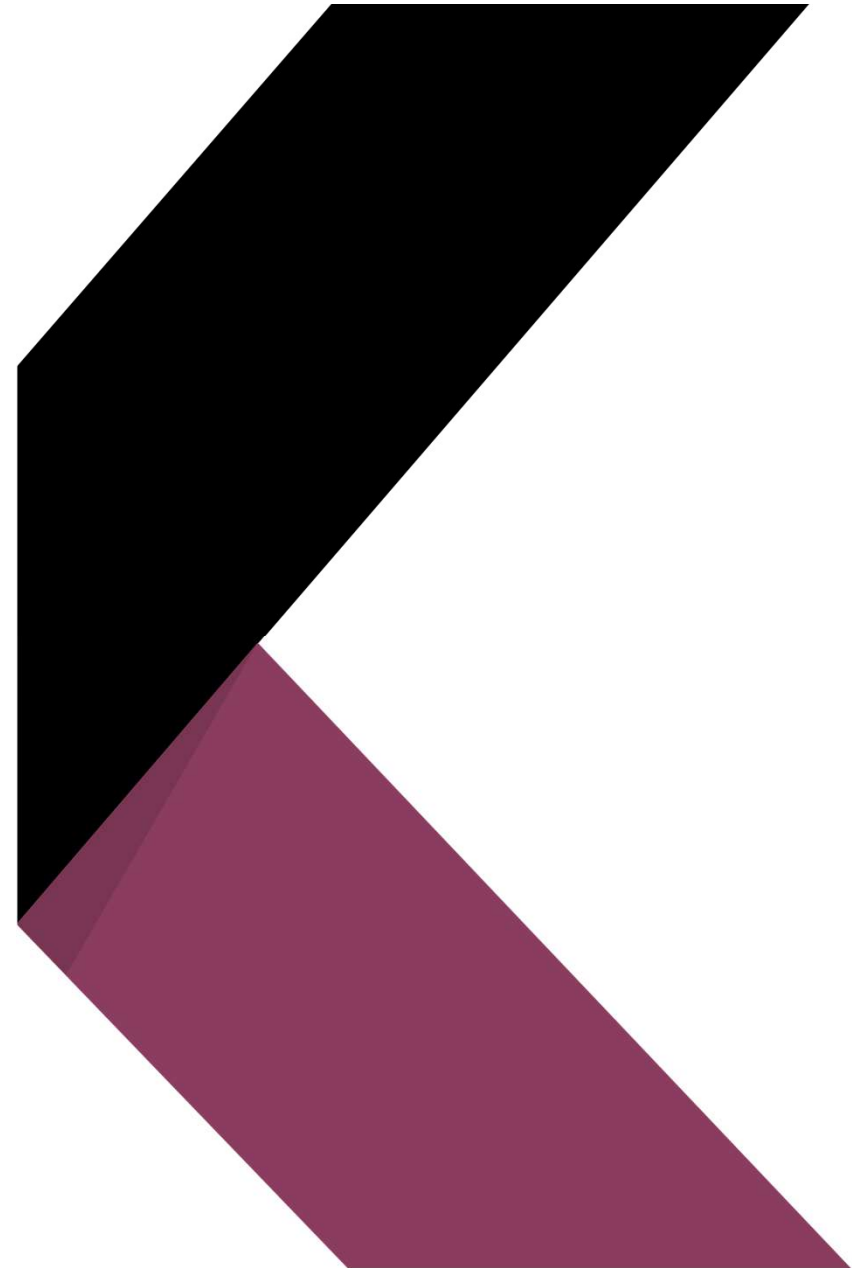


CAPL

# Basic Guide





# CAN Access Programming Language

## Introduction

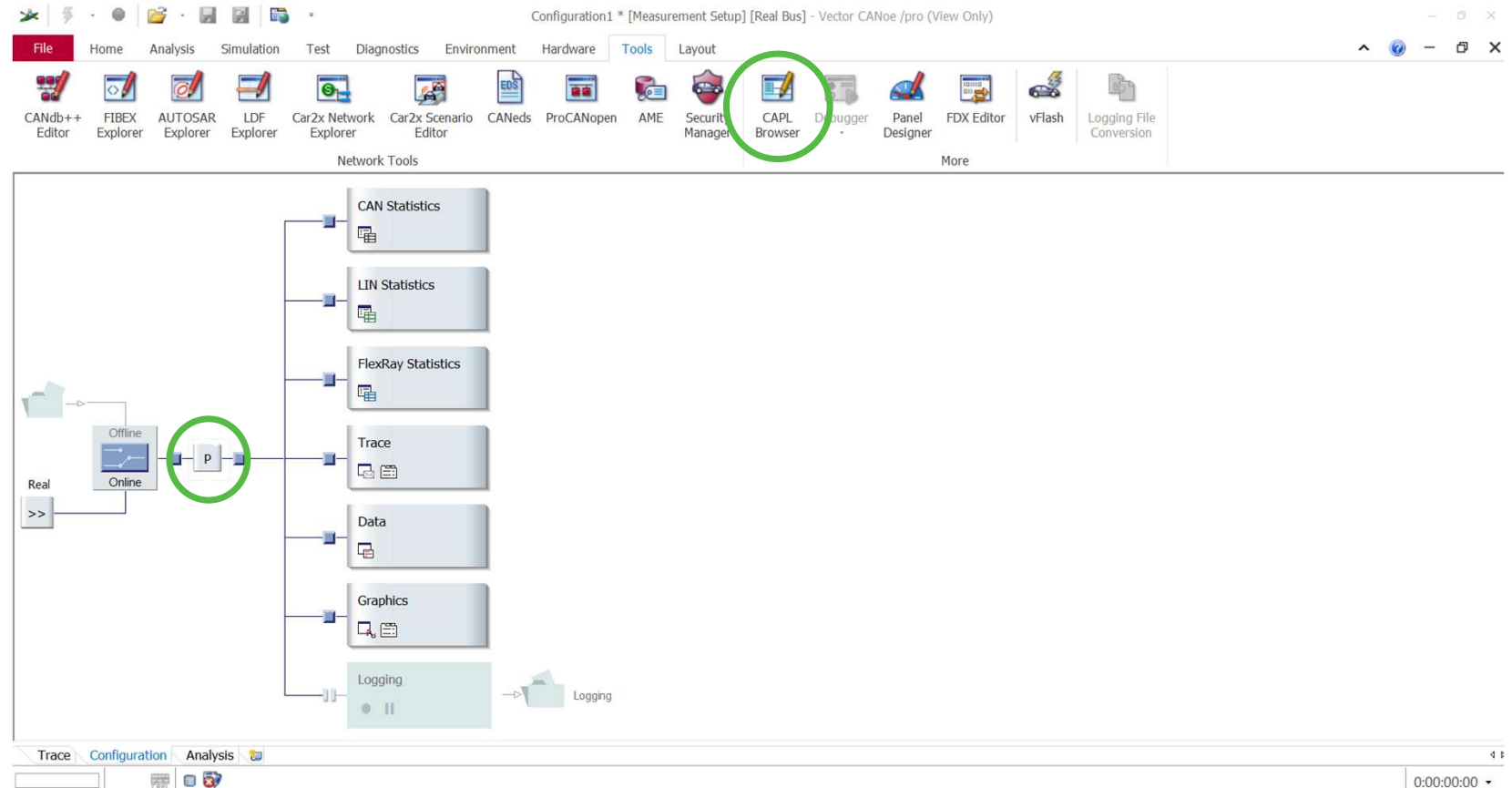
### CAPL

- The CAN Access Programming Language (CAPL) is a C-Like programming language, which allows you to quickly develop codes that make CANalyzer or CANoe simulations more powerful.
- It is a procedural language whereby the execution of program blocks (*Event Procedures*) is controlled on events.
- The code defined in the event procedures is executed when an event occurs, i.e. you can send a message on the bus in response to a “key press” (*on key*), track the occurrence of messages on the bus (*on message*), or execute certain actions cyclically (*on Timer*).



# CAPL Programming

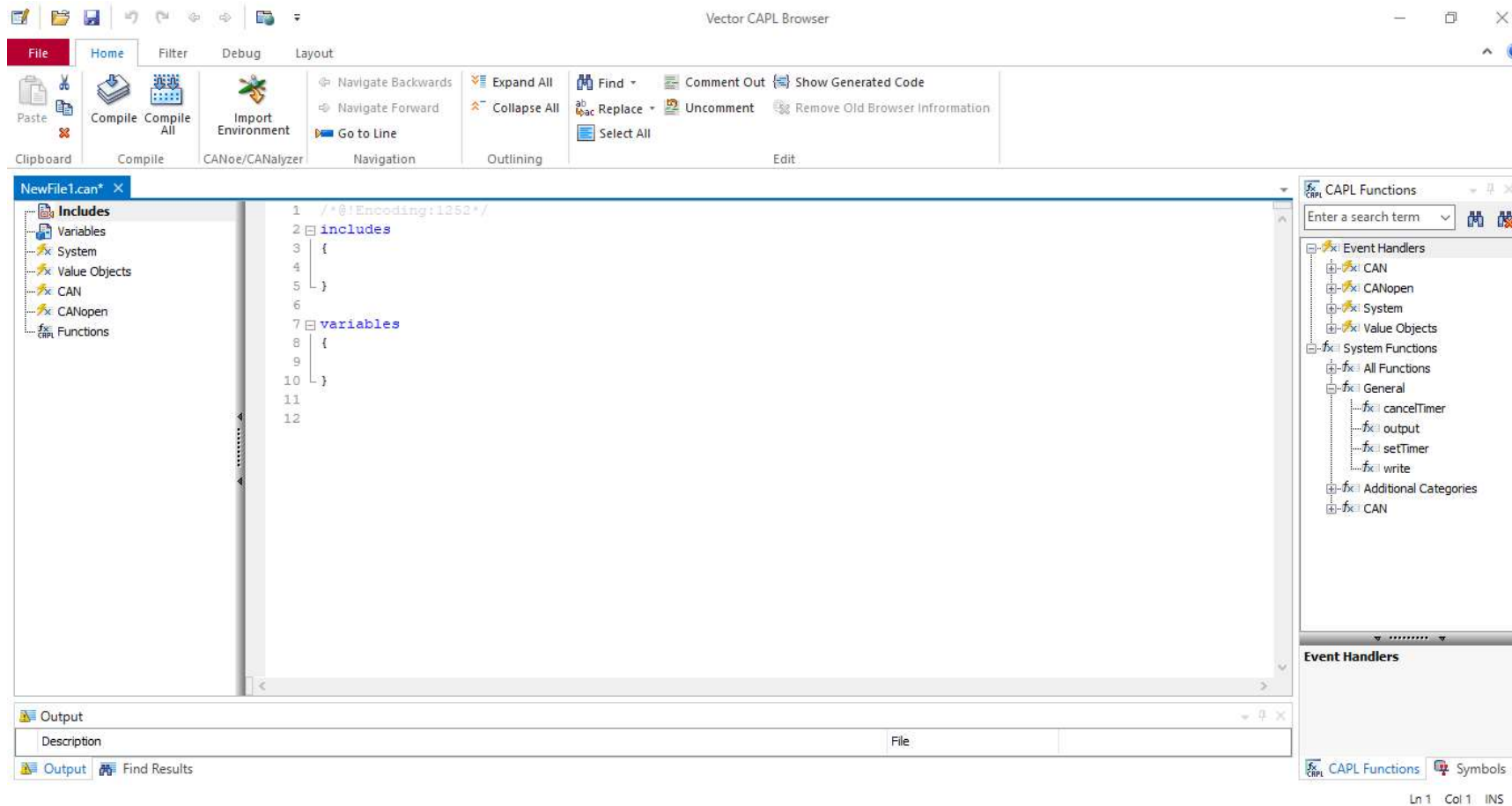
- It is possible to access *CAPL browser* directly from CANalyzer or CANoe through the section **Tools**...
- ... or with a double click on the *Program Node* in **Configuration**.





# CAPL Programming

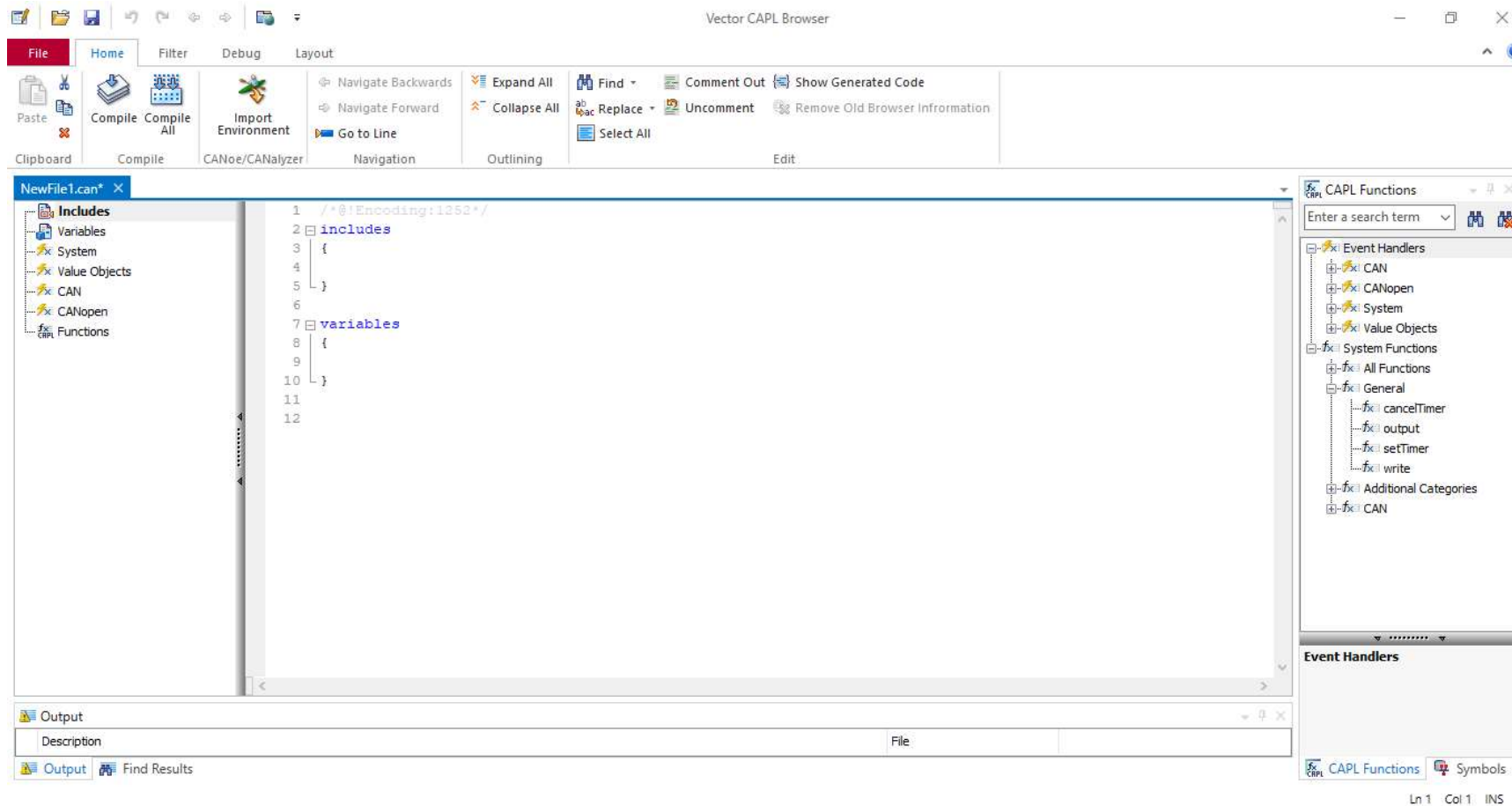
- On the left side, you can find a 'tree' with all the defined variables.
- On the right side, you have the possibility to drag functions or events in the script. You can drag Network messages/signals too (Symbol tab).





# CAPL Programming

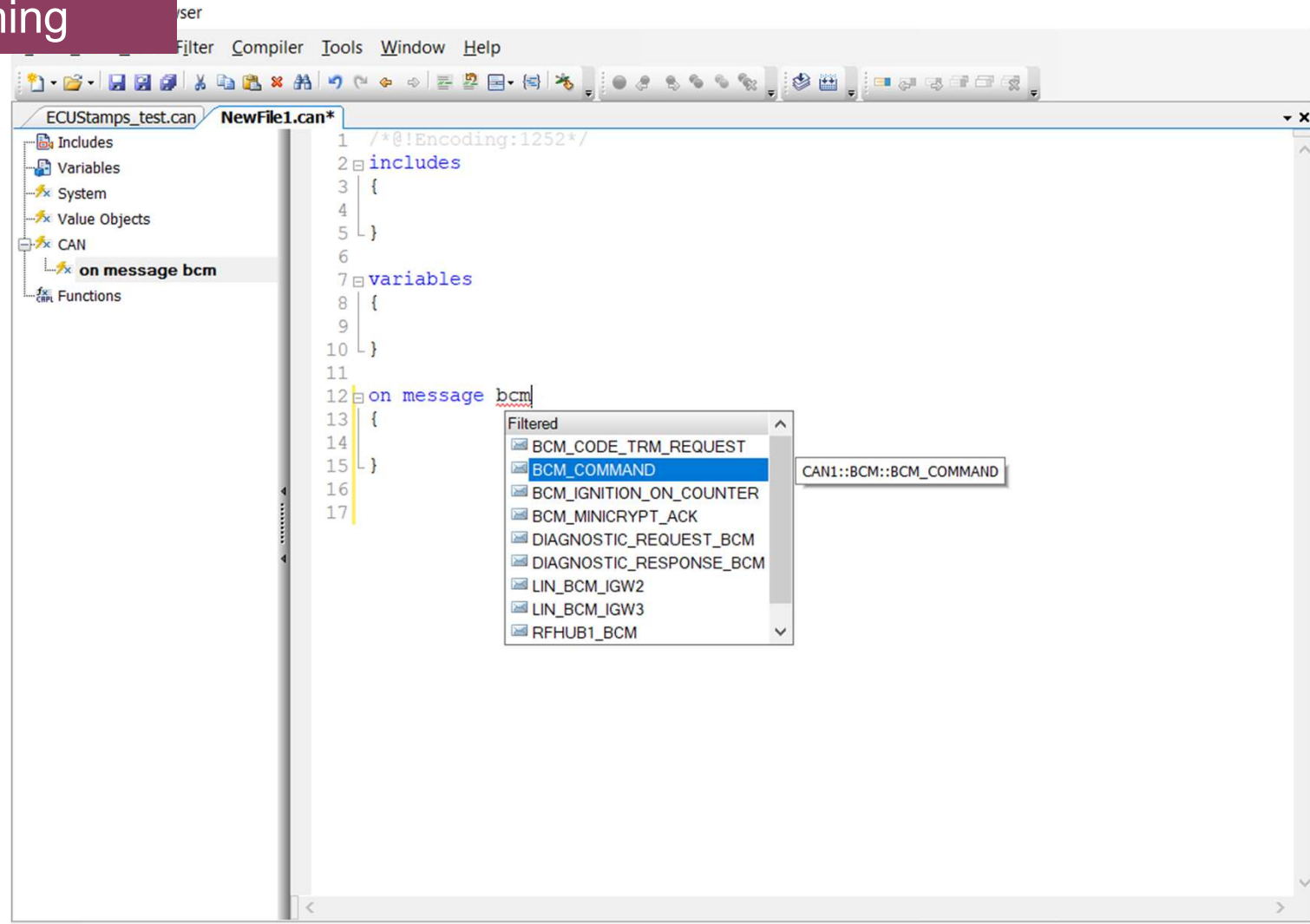
- The central window is where you can write your code.
- When you open a new script, CAPL browser will show you a blank script, with only the two sections: *include* and *variables*.
- After the variables definition it is possible to write the code.





# CAPL Programming

Once you have configured CANalyzer or CANoe and you have linked a Database CAN (.dbc), CAPL browser can handle messages directly from the database.





## CAPL Structure

A CAPL code consists of three distinct parts:

- Global Variable Declarations
- Event Procedures
- User Defined Functions





## CAPL Variables

Data Types available in CAPL are:

- Integers variables: *int*, *int64*, *dword*, *word*, *char*, *byte*
- Floating point numbers: *float* and *double*
- CAN messages and Timers

All variables can be initialized in their declarations, except for the timer. If a variable is not initialized, the compiler will initialize the variable with default values (unless otherwise defined : 0 ).

- CAPL permits the declaration of arrays, analogous to their declaration in the C programming language.



## CAPL Timers

Variables of the type *Timer* (based on seconds) or *msTimer* (based on milliseconds) need to be initialized. They serve to generate time events.

These must be initialized with a proper function which is ***setTimer()***.

i.e. **variables**

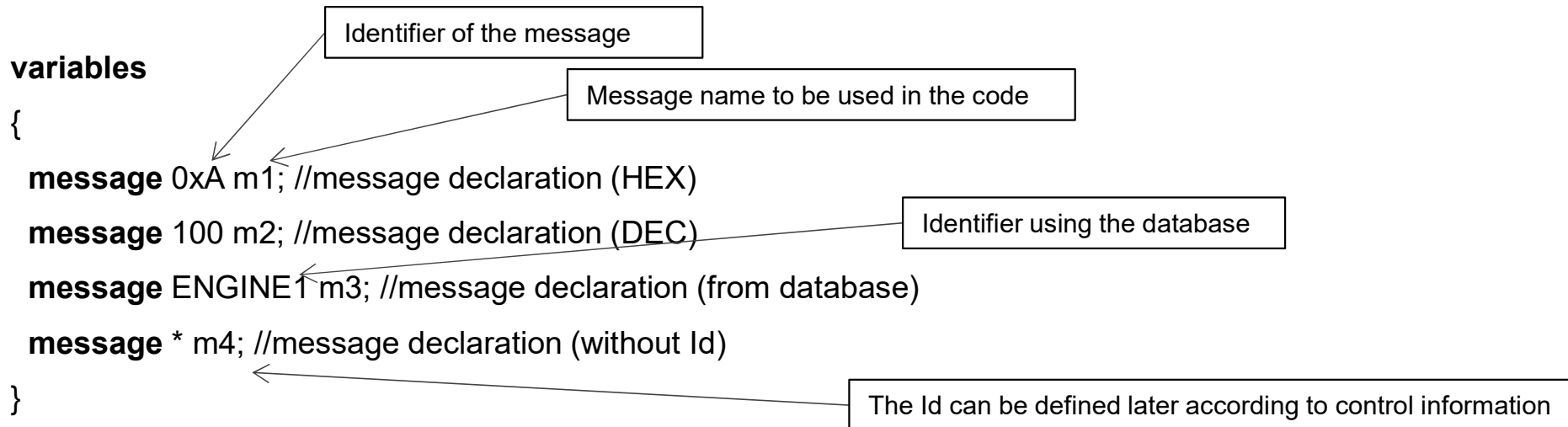
```
{  
  timer t;  
  msTimer t1;  
}  
  
on start  
{  
  setTimer(t,10); // set timer to 10 seconds  
  setTimer(t1,100); //set timer to 100 milliseconds  
}
```



## CAPL Messages declaration

Messages can be defined with the key word **message**.

The complete declaration includes *message identifier* or the *message name*, when a database has been linked to the configuration.





## CAPL Messages initialization

It is possible to access control information for CAN message objects using the following component selector:

**on start**

```
{  
  m1.id = 0xFA; // Message Identifier  
  m1.dlc = 4; // Data Length Code  
  m1.dir = rx; // Direction of transmission (rx: received, or tx: transmitted)  
  m1.rtr = 0; // Remote Transmission Request (0: no RTR, or 1: with RTR)  
  m1.time = (10,microseconds); // Time point, units  
}
```



## CAPL Event Procedure

The main event procedure you can use in CAPL are:

- Receipt a CAN message **on message {}**
- Press of a key **on key {}**
- Initialization of a measurement **on preStart {}**
- Measurement start **on start {}**
- End of measurement **on stopMeasurement {}**
- CAN controller goes to ErrorActive **on errorActive {}**
- CAN controller goes to ErrorPassive **on errorPassive {}**
- CAN controller goes to Bus Off **on busOff {}**
- Elapse of a timer **on timer {}**
- Occurrence of an error frame **on errorFrame {}**
- Environment variable change **on envVar {}**, only for CANoe (the new value is acquirable by using the command **getValue(this)**)



## CAPL React to Messages

The event procedure type **on message** is provided to react to receipt of CAN messages in the CAPL nodes:

- **On message** 123 // React to message considering the ID (dec) only  
{ ...  
}
- **On message** 0x123 // React to message considering the ID (hex) only  
{ ...  
}
- **On message** ENGINE1 // React to message the name of the message only  
{ ...  
}
- **On message** CAN1.123 // React to message considering the chip (dec)  
{ ...  
}



## CAPL React to Messages

The event procedure type on message is provided to react to receipt of CAN messages in the CAPL nodes:

- **On message \*** // React to all messages of all the buses  
    { ...  
    }
- **On message CAN2.\*** // React to all messages of a single network  
    { ...  
    }
- **On message 100-200** // React to all messages with identifiers between 100 and 200  
    { ...  
    }



## CAPL Example

React to messages:

- variables
- {
- message CAN1.34 sendMsg1;
- }
  
- on message CAN1.34
- {
- message CAN2.34 sendMsg2;
- sendMsg2 = this;
- sendMsg2.byte(4)=0;
- output(sendMsg2);
- }





## CAPL React to Keyboard events

With on key procedures you can execute certain actions by key press:

- **on key 'a'** // React to press of .a. key  
{ ...  
}
- **on key ' '** // React to press of spacebar  
{ ...  
}
- **on key 0x20** // React to press of spacebar  
{ ...  
}
- **on key F1** // React to press of F1 key  
{ ...  
}



## CAPL React to Keyboard events

With on key procedures you can execute certain actions by key press:

- **on key** ctrlF12 // React to press of CTRL-F12  
    { ...  
    }
- **on key** PageUp // React to press of Page Up  
    { ...  
    }
- **on key** Home // React to press of Home  
    { ...  
    }
- **on key** \* // React to any key press  
    { ...  
    }



## CAPL Example

React to Keyboard events:

variables

```
{  
  int counter = 0;  
}  
  
on key 'a'  
{  
  write("A total of %d messages 0x1A1 counted",counter);  
}  
  
....
```

```
....  
on message 0x1A1  
{  
  counter+=1;  
  output(this);  
}  
on message *  
{  
  output(this);  
}
```



## CAPL React to Time events

Once you have created a timer and it is expired, the function developed in the event **on Timer** is executed.

- You can use this functionality to develop cyclic actions, or to respond to an event after a delay.
- The function **setTimer()** needs 2 parameters, the name of the timer and the length. The maximum length varies according to the specified unit.
- The function **setTimerCyclic()** allows to execute cyclic actions without resetting the timer.
- Defining the timer as *timer*, it will be calculated each second, so the maximum value that can be reached is 1799 seconds.
- Otherwise, using *msTimer* to define the timer, it will be calculated each millisecond, so the maximum value available is 65535 milliseconds.



## CAPL Example

```
variables
{
  msTimer myTimer;
  message 100 msg;
}

on key 'a'
{
  setTimer(myTimer,20);
}

on timer myTimer
{
  output(msg);
}
```

```
variables
{
  msTimer myTimer;
  message 100 msg;
}

on key 'a'
{
  setTimerCyclic(myTimer,20); //this functionality allows to make cyclic actions
}

on timer myTimer
{
  output(msg);
}
```



## CAPL React to System Events (*preStart*, *Start*, *stopMeasurements*)

These events are used to perform actions before, at the start of, and after CANalyzer or CANoe measurements.

- They can be called just once in the code
- The event *on preStart* is associated with the press of the “GO” button in CANalyzer or CANoe. It is possible to initialize variables, read data from files or write something on the “write” window. Here, it is impossible to send CAN messages onto the bus or set timers.
- After the *preStart*, all the actions in the *on start* can be performed. Here you can set Timers or send messages onto the bus, and initialize environmental variables. The measurement is also started at this time.
- When you press the “STOP” button, the *on stopMeasurements* event is executed. This way, you can print statistics in the write window or output messages onto the bus or write a log file. After this event is finished the measurement will be stopped.



## CAPL React to CAN controller Events

*On errorActive, on errorPassive, on warningLimit, or on busOff.*

- These event procedures are called during a state transition or in response to a change in the CAN controller's error counter.
- These are useful to monitor the error counter, to terminate measurements if necessary, or to execute a reset after a transition to the Bus-Off state.
- The CAN controller's error counter can be accessed using the key word **this.errorCountTX** or **this.errorCountRX**.

**on errorPassive**

```
{  
  write("CAN controller is in errorPassive");  
  write("errorCountTX = %d",this.errorCountTX);  
}
```



## CAPL Expressions in CAPL

CAPL syntax is based on the C programming language. The following expressions are permitted as they are in C:

- Instruction blocks: {...}
- If (condition) {...}
- If (condition){...} else {...}
- Switch, case, default
- For ..., while ..., do ... while loops
- Continue and break
- Return





## CAPL Key word: *this*

The key word ***this*** is used to refer to the data structure of an object within an event procedure for receiving a CAN object or an environment variable.

### ***Example:***

```
on message 100
{
    byte byte_0;
    byte_0 = this.byte(0);
    ...
}
```



### ***Example:***

```
on value_change Variable1
{
    write ("Variable1 has this value now: %d", this.value);
    ...
}
```





## Example

## Body control module emulation 1/4

```
includes
{
}

variables
{
    message BCM_COMMAND bcm;
    message SWC swc;
    int ISWCM = 0; // Infotainment Steering Wheel Controls Management not enabled
}
```



## Example

## Body control module emulation 2/4

```
on message BCM_COMMAND
{
  if ((this.OperationalModeSts==4)|| // Ignition On
      (this.OperationalModeSts==5)|| // Ignition Pre Start
      (this.OperationalModeSts==6)|| // Ignition Start
      (this.OperationalModeSts==7)|| // Ignition Cranking
      (this.OperationalModeSts==8)) // Ignition On Engine On
  {
    ISWCM = 1; // Infotainment Steering Wheel Controls Management enabled
  }
  else
  {
    ISWCM = 0; // Infotainment Steering Won envVar Command_11Sts_env // OK button
  }
}
```



## Example

## Body control module emulation 3/4

```
on envVar Command_11Sts_env // OK button
{
  if (ISWCM==1) // Infotainment Steering Wheel Controls Management enabled
  {
    swc.Command_11Sts = @this;
    output (swc);
  }
  else {} // Infotainment Steering Wheel Controls Management not enabled
}
on envVar Command_12Sts_env // UP button
{
  if (ISWCM==1) // Infotainment Steering Wheel Controls Management enabled
  {
    swc.Command_12Sts = @this;
    output (swc);
  }
  else {} // Infotainment Steering Wheel Controls Management not enabled
}
```

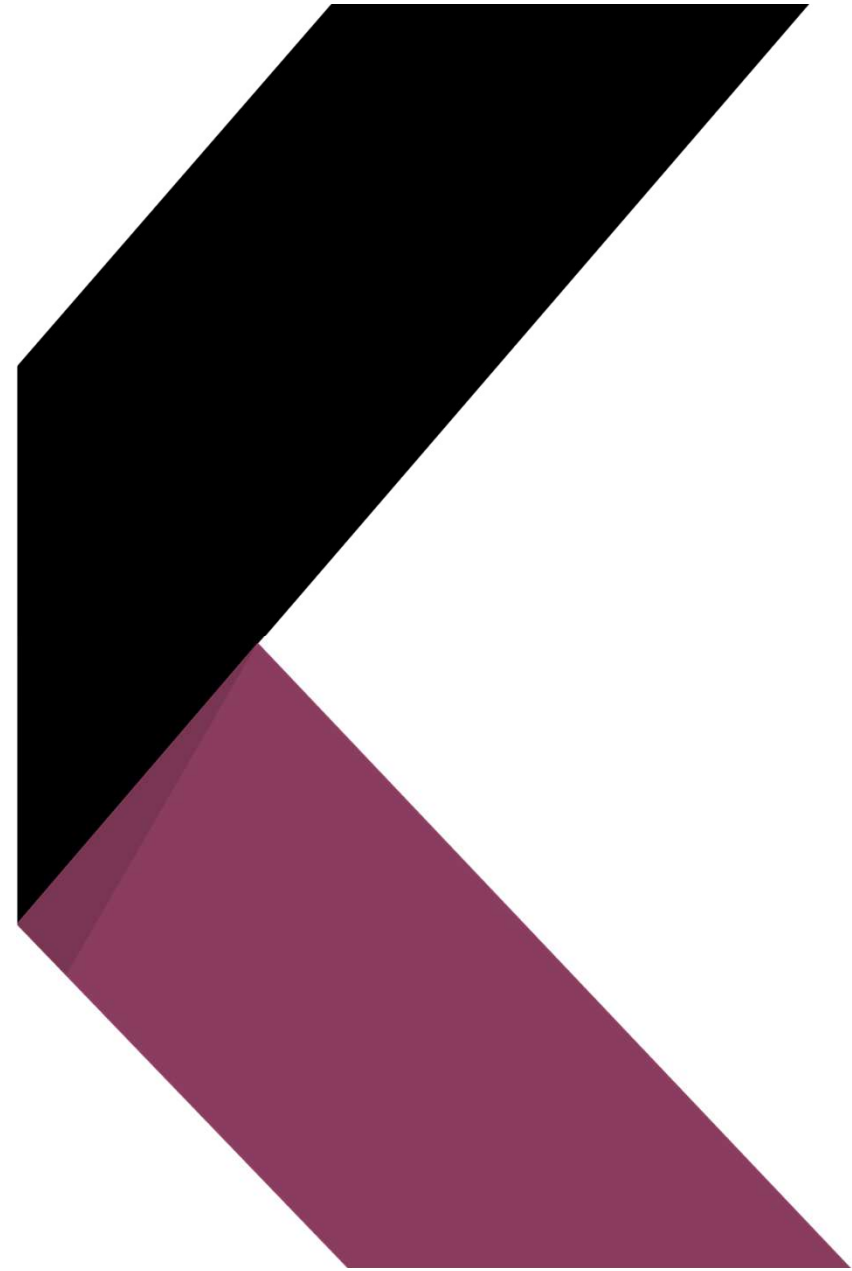


## Example

## Body control module emulation 4/4

```
on envVar Command_13Sts_env // DOWN button
{
  if (ISWCM==1) // Infotainment Steering Wheel Controls Management enabled
  {
    swc.Command_13Sts = @this;
    output (swc);
  }
  else {} // Infotainment Steering Wheel Controls Management not enabled
}
on envVar Command_14Sts_env // RIGHT button
{
  if (ISWCM==1) // Infotainment Steering Wheel Controls Management enabled
  {
    swc.Command_14Sts = @this;
    output (swc);
  }
  else {} // Infotainment Steering Wheel Controls Management not enabled
}
```

# IG BLOCK and CAPL





## INTRODUCTION

During V&V activities on both vehicle and HIL, sometimes it is necessary to change signals that ECUs send or receive on CAN bus. In some cases, the maneuvers to vary these signals aren't suitable to be replicated physically due to:

- Dangerous conditions for the driver
- Damages to vehicle components (sensors, actuators, wirings, ECUs)
- Difficult vehicular conditions
- Long-time or particular testing conditions

Others application cases could be:

- Loss of Communication (LoC) between real ECUs
- MC (Message Counter) and CRC (Cyclic Redundancy Check) errors (Implausibility Faults)

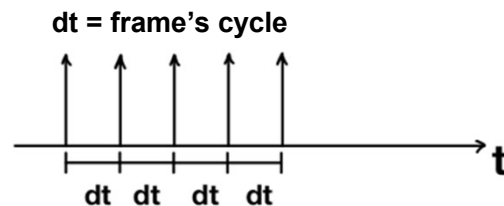
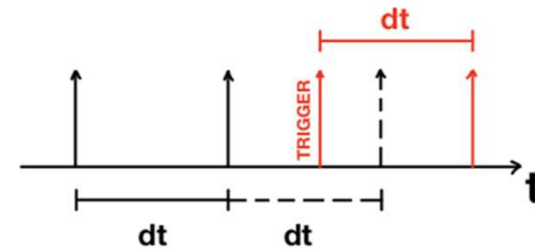
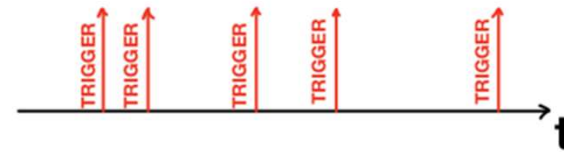
**IG BLOCK and CAPL can be used for all those purposes**



# INTRODUCTION

On CAN Bus, different types of signals can be sent and received from ECUs:

- On Event (e.g. Diagnostic Frames)
- Cyclic On Event (e.g. Impact Frames)
- Cyclic



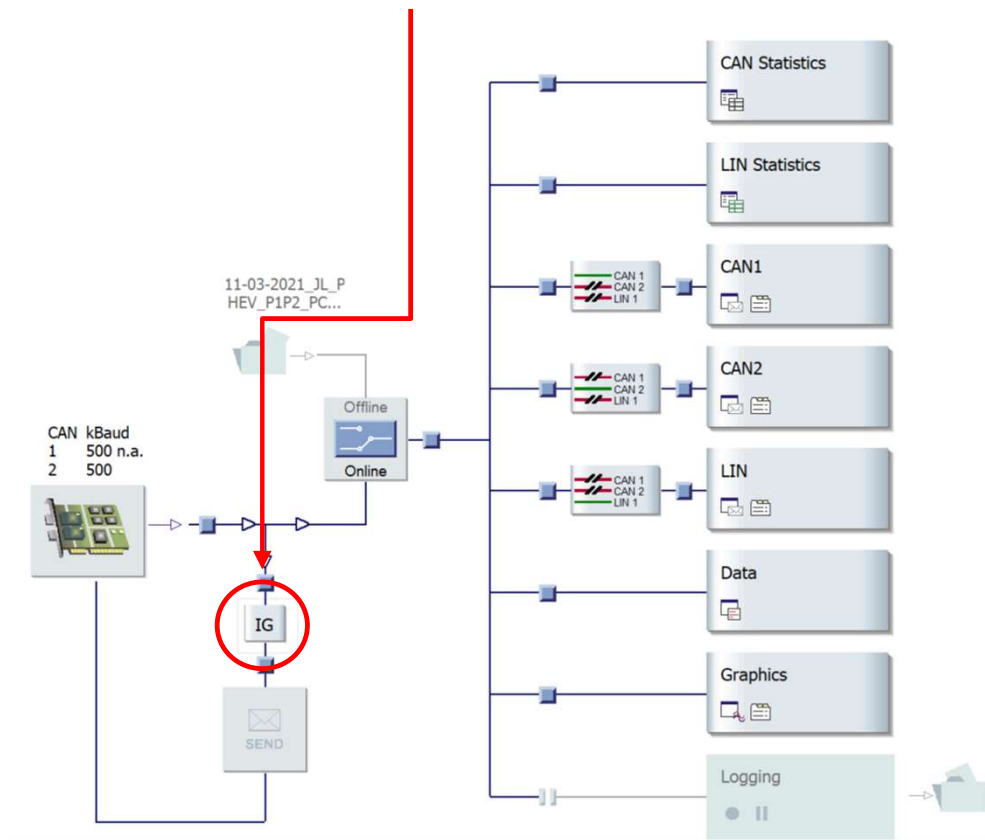
According to signals' type, we can choose which type of gateway to be used



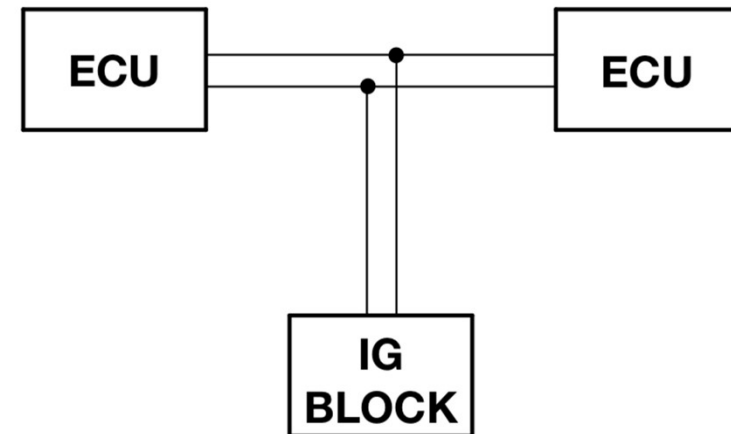


## IG BLOCK

The Interactive Generator Block (IG) is used to create and send **On Event** messages ONLY.



**WIRINGS' SCHEME: T-CONNECTION**  
The IG node is in parallel with the CAN bus





## IG BLOCK

For this purpose, both a list of messages and trigger conditions are the input in a configuration dialog. All the signals included in the selected messages are listed too and their values can be freely changed, according to the associated CAN database (DBC of the architecture's project under testing). The main characteristic of the Interactive Generator Block is the possibility to configure and interactively send messages during a running measurement (online).

Signals of the selected message are shown in the signal list

**Transmitted frame list**

| Message Name                   | Mag Params | Send | Triggering | Data Field                              |   |   |   |   |   |   |   |   |
|--------------------------------|------------|------|------------|-----------------------------------------|---|---|---|---|---|---|---|---|
|                                | Channel    | DLC  |            | Cycle Time [ms]                         | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| TrainingNetwork:fmSensorResult | CAN 1      | 3    | now        | <input type="checkbox"/> 1              | 0 | 0 | 0 |   |   |   |   |   |
| TrainingNetwork:fmKey8         | CAN 1      | 4    | now        | <input type="checkbox"/> 10             | 0 | 0 | 0 | 0 |   |   |   |   |
| TrainingNetwork:fmKey12        | CAN 1      | 1    | now        | <input checked="" type="checkbox"/> 10  | 0 |   |   |   |   |   |   |   |
| TrainingNetwork:fmLamps        | CAN 1      | 1    | now        | <input checked="" type="checkbox"/> 123 | 0 |   |   |   |   |   |   |   |
| TrainingNetwork:fmSensorA      | CAN 1      | 8    | now        | <input type="checkbox"/> 250            | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| TrainingNetwork:fmSensorB      | CAN 1      | 8    | now        | <input checked="" type="checkbox"/> 250 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| TrainingNetwork:fmStatus       | CAN 1      | 4    | now        | <input type="checkbox"/> 200            | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Signal list**

| SB | Signal Name | Raw Value                             | Phys Value      | Unit | Dec | Phys Step    | Inc | Wave form |
|----|-------------|---------------------------------------|-----------------|------|-----|--------------|-----|-----------|
| 8  | sigRotKnob  | 0                                     | 0               | %    | -   | 4.9853372434 | +   | None      |
| 7  | sigKey8     | <input checked="" type="checkbox"/> 1 | Key pressed     |      | -   | 1            | +   | None      |
| 6  | sigKey7     | <input checked="" type="checkbox"/> 1 | Key pressed     |      | -   | 1            | +   | None      |
| 5  | sigKey6     | <input type="checkbox"/> 0            | Key not pressed |      | -   | 1            | +   | None      |
| 4  | sigKey5     | <input checked="" type="checkbox"/> 1 | Key pressed     |      | -   | 1            | +   | None      |
| 3  | sigKey4     | <input type="checkbox"/> 0            | Key not pressed |      | -   | 1            | +   | None      |
| 2  | sigKey3     | <input type="checkbox"/> 0            | Key not pressed |      | -   | 1            | +   | None      |
| 1  | sigKey2     | <input type="checkbox"/> 0            | Key not pressed |      | -   | 1            | +   | None      |
| 0  | sigKey1     | <input type="checkbox"/> 0            | Key not pressed |      | -   | 1            | +   | None      |



## IG BLOCK

- For each message, its parameters such as ID and DLC can be seen in the configuration window
- The trigger conditions (Triggering) specify the instant of time the message will be sent
- The content of the message (i.e. signals) can be changed in the Data Field (hex values) or in the Signal list related to the frame (hex values or physical values)

| SB  | Signal Name      | Raw Value                  | Phys Value   | Unit      | Dec | Phys Step   | Inc | Wave form generation | Modification                               |
|-----|------------------|----------------------------|--------------|-----------|-----|-------------|-----|----------------------|--------------------------------------------|
| 168 | ThrottlePosition | 0                          | 0            | %         | -   | 5.096       | +   | None                 | Define... <input type="checkbox"/> enabled |
| 80  | RawPVS2          | 0                          | 0            | V         | -   | 0.254901959 | +   | None                 | Define... <input type="checkbox"/> enabled |
| 88  | RawPVS1          | 0                          | 0            | V         | -   | 0.254901959 | +   | None                 | Define... <input type="checkbox"/> enabled |
| 152 | ACPressure       | 1FF                        | SNA          | bar       | -   | 2           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 104 | VehicleSetSpeed_ | 0                          | 0            | km/h(mph) | -   | 13          | +   | None                 | Define... <input type="checkbox"/> enabled |
| 251 | TransWarmUpReq   | <input type="checkbox"/> 0 | WarmUp_Not_E | -         | -   | 1           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 96  | TransWarmUpMin   | 0                          | 0            | rpm       | -   | 325         | +   | None                 | Define... <input type="checkbox"/> enabled |
| 249 | ThermalManagem   | <input type="checkbox"/> 0 | Not_active   | -         | -   | 1           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 200 | RawTPS2          | FF                         | 4.999999965  | V         | -   | 0.254901959 | +   | None                 | Define... <input type="checkbox"/> enabled |
| 192 | RawTPS1          | FF                         | 4.999999965  | V         | -   | 0.254901959 | +   | None                 | Define... <input type="checkbox"/> enabled |
| 72  | MessageCounter_E | 0                          | 0            | -         | -   | 1           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 255 | Lv1CrsCtrlOff    | <input type="checkbox"/> 0 | 0            | -         | -   | 1           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 254 | ESP_TrqRqEnabl   | <input type="checkbox"/> 0 | 0            | -         | -   | 1           | +   | None                 | Define... <input type="checkbox"/> enabled |
| 128 | EngMaxRPM_W      | 0                          | 0            | -         | -   | 500         | +   | None                 | Define... <input type="checkbox"/> enabled |
| 64  | CRC_ENGINE1      | 0                          | 0            | -         | -   | 13          | +   | None                 | Define... <input type="checkbox"/> enabled |

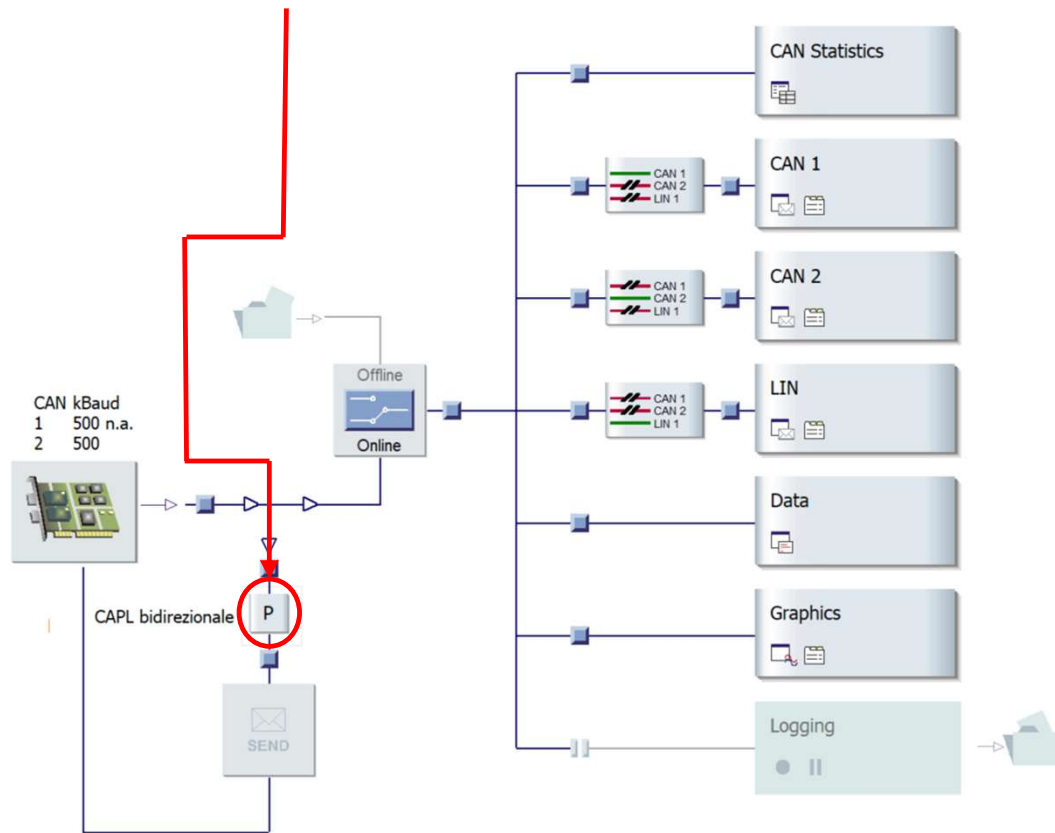
Default input mode: CAN

Close Help



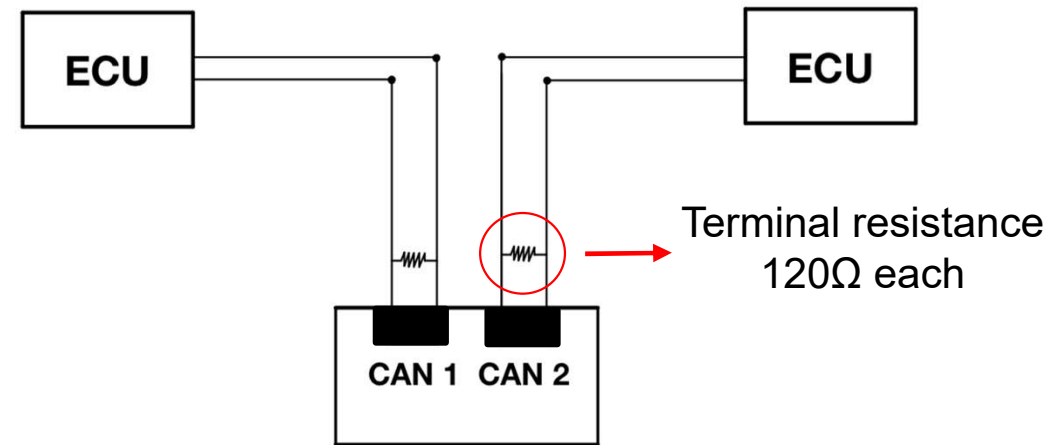
# CAPL

The CAPL is used to generate and send **Cyclic and Cyclic On Event** messages.



## WIRINGS' SCHEME: S-CONNECTION

We need to physically interrupt the CAN bus





# CAPL

## 1. Basic CAPL code : used to overturn messages from a port to the other (no frame modification)

```
on message CAN2.*           //Focus on CAN2 port on CANcase
{
    message CAN1.* msg;      //Take the message on CAN1 port which is passing through the channel

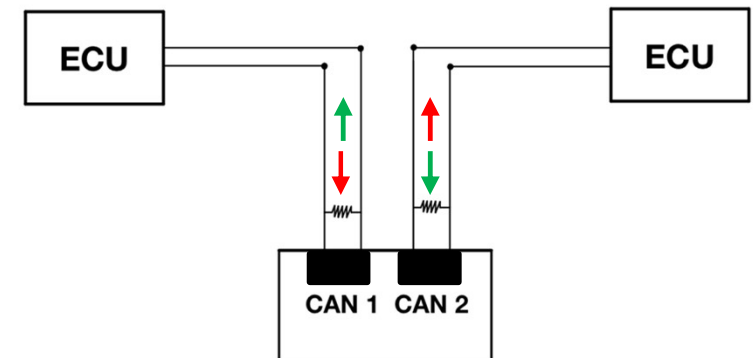
    if(this.dir != rx) return; //If this message is transmitted by CAN1 port out from CANcase, exit from the function
    output(msg);              //If the message is received by CAN1 port, send it out from CANcase using CAN2 port

    return;
}

on message CAN1.*           //Focus on CAN1 port on CANcase
{
    message CAN2.* msg;      //Take the message on CAN2 port which is passing through the channel

    if(this.dir != rx) return; //If this message is transmitted by CAN2 port out from CANcase, exit from the function
    output(msg);              //If the message is received by CAN2 port, send it out from CANcase using CAN1 port

    return;
}
```





# CAPL

## 2. Advanced CAPL code : used to change the signal's value of a specific message

```
2 variables
3 {
4   int i, i_byte, i_bit, crc_reg, X, sign=0;
5   dword crc_resto, poly;
6   int array_bytes[8];
7 }
8
9
10 on key '1' //Pressing button "1" on keyboard we set variable X equal to 1
11 {
12   X = 1;
13 }
14 on key '0' //Pressing button "0" on keyboard we set variable X equal to 0 (used for reset)
15 {
16   X = 0;
17 }
18
19 on message CAN2.* //Focus on CAN2 port on CANcase
20 {
21
22   message CAN1.* msg; //Take the message on CAN1 port which is passing through the channel
23
24   if(this.dir != rx) return; //If this message is transmitted by CAN1 port out from CANcase, exit from the function
25
26   if(this.id==0x150 & X ==1) //Check if the message ID is equal to 150 hex and X=1
27   {
28     message CAN1.BPCM_MSG_01 msg; //Assign to variable "msg" the message we want to modify
29     msg.HVbatCntctrOpn=1; //Choose signal to be modified inside the message
30     msg.CRC_150h=CRCCalc(msg); //Call the function to calculate the message's CRC
31     output(msg); //Send it in output with changed value
32     msg=this;
33     return;
34   }
35
36   else //If ID is not equal to 150 hex or X!=1 overturn the message only
37   {
38     message CAN1.* msg;
39     msg = this;
40     output(msg);
41     return;
42   }
43 }
```



## COMMON USES OF CAPL

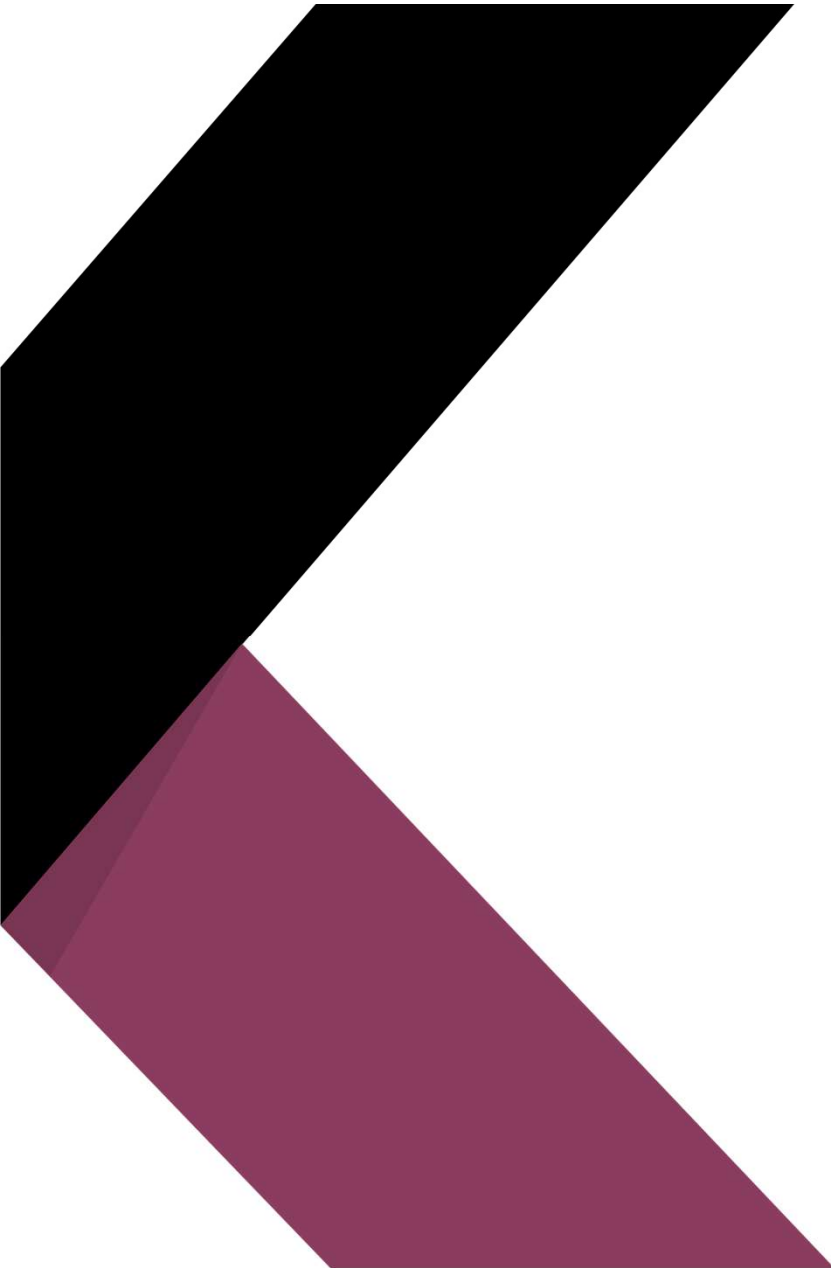
- To change the signal's value of a specific cyclic and cyclic on event message
- To set a Lost of Communication (LoC) between different ECUs
- To set implausible data transmission/reception (MC, CRC, SNA or validity bit)
- To change more signals of the same CAN bus at the same instant
- To change signals belonging to different CAN buses (e.g. C1, ePT) → Double Gateway



## CAPL vs IG BLOCK

| CAPL                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                          | IG BLOCK                                                                                                                                               |                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| ADV                                                                                                                                                                                                                                                                                                                                                   | DISADV                                                                                                                                                                   | ADV                                                                                                                                                    | DISADV                                                                                                                           |
| <ul style="list-style-type: none"><li>• Possibility to change a signal independently from the type of frame it belongs</li><li>• Flexibility and adaptability</li><li>• Possibility to change signals in two different CAN in the same time</li><li>• Wide range of programming (limited only by the coding skills of the testing engineer)</li></ul> | <ul style="list-style-type: none"><li>• Need to write a C++ script</li><li>• Physical interruption of CAN bus is needed</li><li>• BoB (Breakout Box) is needed</li></ul> | <ul style="list-style-type: none"><li>• Both easy and fast to configure and use</li><li>• Doesn't interrupt the CAN (parallel configuration)</li></ul> | <ul style="list-style-type: none"><li>• Possibility to change only message on event (strong limitation on usage cases)</li></ul> |





# CONTACTS

Via G. Porzio,  
Centro Direzionale Isola B3  
80143 Napoli Via G. Porzio

[www.kineton.it](http://www.kineton.it)

